Sentinel 550 build guide

A visual-guide companion covering wiring, flight controller setup, configuration, and the build and test order for the Sentinel 550 quadcopter.

Overview

The Sentinel 550 is a 550 mm quadcopter running ArduCopter on a Pixhawk-class flight controller, with a Raspberry Pi 5 as the companion computer. It carries a Pi camera on a fixed straight-down mount, with a thermal core planned for later. The autonomous scripts in this project run on the Pi. Only the autonomous controller sends flight commands, and every other component (perception, gestures, voice, cloud sync) produces reports or intents that pass through that controller's safety checks.

Safety rule that applies throughout: keep every DRY_RUN flag on until its stage passes, and do not skip stages.

Build specification

Item	Choice	Reason
Frame	550 mm quad, fixed landing gear	Room for payload and a clear camera view
Flight controller	Pixhawk-class, ArduCopter	MAVLink support and mature failsafes
Motors and ESCs	2306-class motors, 30 to 40 A ESCs, 10-inch props	About 1.5 kg payload capacity
Battery	4S LiPo, about 5000 mAh	Balances flight time and weight
Companion computer	Raspberry Pi 5 with active cooling	Runs MAVLink, logging, and light vision
Camera	Pi camera module, fixed nadir mount	Simple geometry, matches default_gimbal_pitch_deg = -90
Thermal (later)	Low-resolution radiometric thermal core	Added after the RGB pipeline works

Item	Choice	Reason
Telemetry	915 or 433 MHz radio pair, per local rules	Ground link
RC	Transmitter with 3-position mode switch, receiver with failsafe output	Manual override is always available
Accessories	Fan on Pi via MOSFET, WS2812 LED strip via level shifter	Power and weather-light scripts

Target all-up weight is about 2.2 kg, with hover around 50 percent throttle.

Wiring

The main connections are below. Diagram: battery feeds the motors and ESCs; the Pixhawk sends PWM to the ESCs, talks to the Pi over UART, talks to the telemetry radio, and receives the RC receiver's failsafe output; the Pi camera connects to the Pi.

From	To	Connection	Notes
4S battery	Motors and ESCs	Main power	Main power distribution
Pixhawk	ESCs	PWM outputs	Motor control signals
Raspberry Pi UART (GPIO 14/15, /dev/ttyAMA0)	Pixhawk SERIAL1	TX and RX cross-connected, shared ground	Matches SERIAL_PORT and 921600 baud in code
Pixhawk SERIAL2	Telemetry radio	Serial	Ground station link
RC receiver	Pixhawk	Failsafe output	Manual override
Pi camera	Raspberry Pi 5	Camera connector	Fixed nadir mount
5 V BEC or UPS HAT	Raspberry Pi 5	Power	Kept separate from ESC rails to avoid brownouts
Pi GPIO via logic-level MOSFET	Fan	PWM, separate supply, flyback diode	Never drive the fan from the GPIO pin directly

From	To	Connection	Notes
Pi GPIO via level shifter	WS2812 LED strip	Data line, separate 5 V supply	Weather status light

Color key used in the wiring diagram: amber for power, blue for control, gray for communications and sensors.

Flight controller parameters

Set these in Mission Planner or QGroundControl, and check them against your firmware version.

```
# Companion computer link (SERIAL1 to Pi)
SERIAL1_PROTOCOL  2        # MAVLink2
SERIAL1_BAUD      921      # 921600 baud

# Ground telemetry (SERIAL2)
SERIAL2_PROTOCOL  2
SERIAL2_BAUD      57

# Failsafes
FS_THR_ENABLE     1        # RC loss: return to launch
FS_GCS_ENABLE     1        # ground link loss: return to launch
FS_BATT_ENABLE    2        # low battery: return to launch
BATT_LOW_VOLT     14.8     # 4S low-voltage warning, adjust to your pack
BATT_CRT_VOLT     14.0     # 4S critical voltage
FS_EKF_ACTION     1        # EKF failure: land

# Geofence
FENCE_ENABLE      1
FENCE_TYPE        3        # altitude and circle
FENCE_RADIUS      50       # meters from home, start small
FENCE_ALT_MAX     15       # meters
FENCE_ACTION      1        # report and RTL

# Speeds for early tests
PILOT_SPEED_UP    100      # cm/s
WPNAV_SPEED       300      # cm/s
```

Adjust BATT_LOW_VOLT and BATT_CRT_VOLT for your pack's cell count and chemistry. Verify every failsafe in the ground station with props off before relying on it.

Perception configuration

Save this as sighting_config.json on the Pi.

```
{
  "drone_id": "sentinel550_01",
  "detector": "yolo",
  "model_path": "yolov8n.pt",
  "camera_source": "0",
  "camera_hfov_deg": 62.0,
  "camera_vfov_deg": 49.0,
  "default_gimbal_pitch_deg": -90.0,
  "mavlink_connection": "/dev/ttyAMA0",
  "mavlink_baud": 921600,
  "telemetry_max_age_s": 1.0,
  "max_range_m": 150.0,
  "confirm_hits": 3,
  "save_snapshots": true,
  "coordinator_url": ""
}
```

Leave coordinator_url empty until the local pipeline is tested. Switch the detector to "thermal" once the thermal core is installed.

Build and test order

Each stage must pass before the next begins. Stages 5 and 6 involve flight.

1. **SITL simulation (ground, no hardware).** Run ArduCopter SITL on a laptop with this parameter set. Confirm telemetry, failsafes, and the geofence.
2. **Bench test, props off.** Connect the Pi, confirm MAVLink telemetry on the Pi, and test each failsafe in the ground station.
3. **Sensors and logging (ground, stationary).** Run the track logger and outbox with the drone stationary.
4. **Perception on recorded footage (offline).** Run the sighting engine on saved video.
5. **Tethered hover (flight).** Short hover on a tether, with the RC transmitter in hand.

6. **Free flight (flight).** Only in a permitted, closed area, with a qualified pilot and the required permissions.

Safety checklist

- Keep DRY_RUN on until each stage passes, and remove one flag at a time.
- Keep the RC transmitter in hand during every flight test, with the mode switch ready to take over.
- Do not connect gesture, voice, or perception output directly to flight. Route all intents through the autonomous controller's checks.
- Verify failsafes before every flight day, not just the first.
- Check local drone regulations for autonomous flight, flight over people, and data collection. Person detection and location reporting usually need permits or official coordination.
- Have a qualified drone pilot or instructor review all flight-related code before any flight. The code is a prototype and has not been flight-tested.

Glossary

- **SITL:** Software-in-the-loop simulation of the flight controller.
- **ESC:** Electronic speed controller, which drives each motor.
- **PWM:** Pulse-width signal sent from the flight controller to the ESCs.
- **MAVLink:** The messaging protocol between the flight controller and companion computer.
- **Geofence:** A boundary the drone must stay within, enforced by the flight controller.
- **RTL:** Return to launch.
- **BEC:** Battery eliminator circuit, which steps battery voltage down to a stable supply.
- **Nadir:** Straight down. The camera points nadir in this build.